

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutakozás**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szelekciós vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A list (*lista*) adattípus

- A *listák* elemek (pozíció szerint) rendezett sorozatai.
- A `list` *mutable* típus: az elemeit megváltoztathatjuk, törölhetjük, beszúrhatunk elemeket, stb.

```
>>> l = [1, None, "Hello", (4, 5), ["Lista", "a", "listában"]]
>>> l.append(2024)
>>> del l[2]
>>> l
[1, None, (4, 5), ["Lista", "a", "listában"], 2024]
>>> l[0] = 0
>>> l[-2] = "A végéről a második"
>>> l
[0, None, (4, 5), "A végéről a második", 2024]
```

A list (*lista*) adattípus műveletei

Operátorok

- egyoperandusú műveletek

`len()` a lista hossza

- kétoperandusú műveletek

`+` összefűzés

`+=` hozzáfűzés

`*` többszörözés

`*=` megtöbbszörözés

`[]` indexelés

`in` tartalmazás

`not in` nem tartalmazás

`del` index szerinti törlés

- többoperandusú műveletek

`[::]` szeletelés

```
len(['egy', 'lista'])  
l = [1] + ['kettő']  
l += [3.0, [4, 'öt']]  
[None] * 10  
l *= 2  
v = l[3]  
l[2] = (2, 'elem')  
3 in l  
'öt' not in l  
del l[1]
```

A list (*lista*) adattípus műveletei

Relációs műveletek

- kétoperandusú műveletek

`==` egyenlő

`!=` nem egyenlő

`<` lexikografikusan kisebb

`>` lexikografikusan nagyobb

`<=` lexikografikusan kisebb
vagy egyenlő

`>=` lexikografikusan nagyobb
vagy egyenlő

```
[1, 2] == [1, 2]
[1, 2] != [2, 1]
[1, 3] < [2, 4]
[3, 1] > [2, 4]
[1, 3] <= [4, 2]
[3, 1] >= [4, 2]
```

Konverzió list (*lista*) típusra

• `list(val)`

- Sztring: a sztring karaktereit sorban tartalmazó lista az eredmény
- Tuple: listát csinál a tuple-ből
- Halmaz: a halmaz elemeit (véletlenszerű sorrendben) tartalmazó lista az eredmény
- Dictionary: a dictionary kulcsait (a beszúrás sorrendjében) tartalmazó lista az eredmény
- Generátor típus: a generátor által visszaadott elemeket a visszaadott sorrendben tartalmazó lista az eredmény

```
>>> list('Hello')
['H', 'e', 'l', 'l', 'o']
>>> list((1,2,3))
[1, 2, 3]
>>> list({'egy', 'kettő', 'három'})
['kettő', 'három', 'egy']
>>> list({1: 'egy', 2: 'kettő'})
[1, 2]
>>> list(range(3))
[0, 1, 2]
```

A list (*lista*) adattípus műveletei

Hasznos műveletek (módosítás)

- módosító metódusok

- `.append()` elem beszúrása a végére
- `.extend()` elemek beszúrása
- `.insert()` elem beszúrása pozícióba
- `.pop()` elem kivétele
- `.remove()` adott értékű elem törlése
- `.reverse()` lista megfordítása
- `.sort()` lista rendezése
- `.clear()` összes elem törlése

```
l.append(42)
l.extend([-1, 0, 1])
l.insert(3, 'negyedik')
ans = l.pop(4)
l.remove('negyedik')
l.reverse()
l.sort()
l.clear()
```


A list (*lista*) adattípus műveletei

Hasznos műveletek (lekérdezés)

- lekérdező metódusok

- `.index()` érték megkeresése
- `.find()` érték megkeresése
- `.rindex()` érték megkeresése
- `.rfind()` érték megkeresése
- `.count()` érték megszámlálása
- `.copy()` lista másolása

```
idx = l.index('első')
idx = l.find('negyedik')
idx = l.rindex('utolsó')
idx = l.rfind('negyedik')
n = l.count(42)
l2 = l.copy()
```

A list (*lista*) adattípus műveletei

Hasznos műveletek (aggregálás)

- függvények

`all()` mind igaz

`any()` van közte igaz

`max()` legnagyobb érték

`min()` legkisebb érték

`sum()` elemek összege

`reversed()` lista bejárása visszafelé

`sorted()` lista elemei rendezve

```
all([True, False, True])
any([True, False, True])
max([5, 3, 2, 4, 1])
min([5, 3, 2, 4, 1])
sum([5, 3, 2, 4, 1])
list(reversed([5, 3, 2, 4, 1]))
sorted([5, 3, 2, 4, 1])
```

A list (*lista*) adattípus műveleteinek tulajdonságai

- A list *mutable* típus. Bizonyos műveletek annak a listának az értékét változtatják meg, amin dolgoznak, és nem új listát hoznak létre!

```
>>> a = b = [1,2,3]
```

```
>>> a is b
```

```
True
```

```
>>> b += [4,5]
```

```
>>> a is b
```

```
True
```

```
>>> a
```

```
[1, 2, 3, 4, 5]
```

```
>>> b
```

```
[1, 2, 3, 4, 5]
```

- Ha egy sztringet mint karakterek sorozatát értelmezzük, akkor beszélhetünk a sztring hosszáról és adott pozícióban lévő karaktereiről.
- Az `s` sztring hosszát (karaktereinek számát) a `len(s)` függvényhívással kaphatjuk meg.
- Az `s` sztring n -dik karakterét az `s[n]` kifejezéssel kaphatjuk meg.
 - A sztringek indexelése 0-tól kezdődik („hanyadik karakter?” helyett „hány karakter előzi meg?”).
 - Az utolsó érvényes index a sztring hossza mínusz 1.
 - Negatív számmal is lehet indexelni, ekkor a sztring végétől előre számolunk („hátról a hányadik karakter?”). A -1 index a sztring utolsó karakterét jelöli ki.
 - Sztring esetén az indexelés művelet egy egy karakter hosszúságú sztringet fog visszaadni (Python-ban ugyanis nincs karakter típus).

A tuple adattípus

- A *tuple*-ök elemek (pozíció szerint) rendezett sorozatai.
- A `tuple` *immutable* típus: ha egyszer létrehoztunk egy tuple-t, nem tudjuk megváltoztatni.
- A `tuple` típus tulajdonképpen egy *immutable list*.
- A `tuple` ennek megfelelően nem rendelkezik a `list` olyan műveleteivel, amelyek megváltoztathatnák az értékét.

```
>>> t = (1, None, "Hello", (4, 5), ["Lista", "a", "tuple-ben"])
>>> del t[2]
TypeError: 'tuple' object doesn't support item deletion
>>> t[2]
'Hello'
```

A tuple adattípus műveletei

Operátorok

- egyoperandusú műveletek
 - `len()` a tuple hossza
- kétoperandusú műveletek
 - `+` összefűzés
 - `+=` hozzáfűzés
 - `*` többszörözés
 - `*=` többszörözés
 - `[]` indexelés
 - `in` tartalmazás
 - `not in` nem tartalmazás
- többoperandusú műveletek
 - `::` szeletelés

```
len(('egy', 'tuple'))
l = (1,) + ('kettő',)
l += (3.0, [4, 'öt'])
(None,) * 10
l *= 2
v = l[3]
l[3][0] = 'négy'
3 in l
'öt' not in l
```

A tuple adattípus műveletei

Relációs műveletek

- kétoperandusú műveletek

`==` egyenlő

`!=` nem egyenlő

`<` lexikografikusan kisebb

`>` lexikografikusan nagyobb

`<=` lexikografikusan kisebb
vagy egyenlő

`>=` lexikografikusan nagyobb
vagy egyenlő

```
(1, 2) == (1, 2)
(1, 2) != (2, 1)
(1, 3) < (2, 4)
(3, 1) > (2, 4)
(1, 3) <= (4, 2)
(3, 1) >= (4, 2)
```

Konverzió tuple típusra

- `tuple(val)`
 - Sztring: a sztring karaktereit sorban tartalmazó tuple az eredmény
 - List: tuple-t csinál a listából
 - Halmaz: a halmaz elemeit (véletlenszerű sorrendben) tartalmazó tuple az eredmény
 - Dictionary: a dictionary kulcsait (a beszúrás sorrendjében) tartalmazó tuple az eredmény
 - Generátor típus: a generátor által visszaadott elemeket a visszaadott sorrendben tartalmazó tuple az eredmény

```
>>> tuple('Hello')
('H', 'e', 'l', 'l', 'o')
>>> tuple([1,2,3])
(1, 2, 3)
>>> tuple({'egy', 'kettő', 'három'})
('három', 'kettő', 'egy')
>>> tuple({1:'egy', 2:'kettő'})
(1, 2)
>>> tuple(range(3))
(0, 1, 2)
```


A tuple adattípus műveletei

Hasznos műveletek (lekérdezés)

- metódusok

- `.index()` érték megkeresése
- `.find()` érték megkeresése
- `.rindex()` érték megkeresése
- `.rfind()` érték megkeresése
- `.count()` érték megszámlálása

```
idx = l.index('első')  
idx = l.find('negyedik')  
idx = l.rindex('utolsó')  
idx = l.rfind('negyedik')  
n = l.count(42)
```

A tuple adattípus műveletei

Hasznos műveletek

- függvények

`all()` mind igaz

`any()` van közte igaz

`max()` legnagyobb érték

`min()` legkisebb érték

`sum()` elemek összege

`reversed()` tuple bejárása visszafelé

`sorted()` tuple elemei rendezve
listaként

```
all((True, False, True))
any((True, False, True))
max((5, 3, 2, 4, 1))
min((5, 3, 2, 4, 1))
sum((5, 3, 2, 4, 1))
tuple(reversed((5, 3, 2, 4, 1)))
sorted((5, 3, 2, 4, 1))
```

A tuple adattípus műveleteinek tulajdonságai

- A `tuple` *immutable* típus. Az egyes műveletek nem annak a tuple-nek az értékét változtatják meg, amin dolgoznak, hanem egy új tuple-t adnak vissza!

```
>>> a = b = (1,2,3)
```

```
>>> a is b
```

```
True
```

```
>>> b += (4,5)
```

```
>>> a is b
```

```
False
```

```
>>> a
```

```
(1, 2, 3)
```

```
>>> b
```

```
(1, 2, 3, 4, 5)
```

- Ha egy sztringet mint karakterek sorozatát értelmezzük, akkor beszélhetünk a sztring hosszáról és adott pozícióban lévő karaktereiről.
- Az `s` sztring hosszát (karaktereinek számát) a `len(s)` függvényhívással kaphatjuk meg.
- Az `s` sztring n -dik karakterét az `s[n]` kifejezéssel kaphatjuk meg.
 - A sztringek indexelése 0-tól kezdődik („hanyadik karakter?” helyett „hány karakter előzi meg?”).
 - Az utolsó érvényes index a sztring hossza mínusz 1.
 - Negatív számmal is lehet indexelni, ekkor a sztring végétől előre számolunk („hátról a hányadik karakter?”). A -1 index a sztring utolsó karakterét jelöli ki.
 - Sztring esetén az indexelés művelet egy egy karakter hosszúságú sztringet fog visszaadni (Python-ban ugyanis nincs karakter típus).

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Az str (sztring) adattípus műveletei

Gyakoribb/hasznos műveletek (Listaműveletek)

- Darabolás érték alapján
list az eredmény
 `.splitlines()` sorokra darabolás
 `.split()` feldarabolás határolók mentén
 `.rsplit()` feldarabolás határolók mentén jobbról
- Összefűzés
str az eredmény
 `.join()` lista elemeinek összefűzése

```
s.splitlines()  
s.split()  
s.rsplit(",")  
  
s = ', '.join(["alma", "körte"])
```

Szeletek (slice)

list, tuple és str típusok esetében

- Egy adott lista-szerű (elemeket adott sorrendben tartalmazó) érték (index szerinti) részlistáját Python-ban szeletek segítségével kapjuk meg.
- Az 1 lista (tuple vagy sztring) n -dik elemétől m -dik eleméig (m -et **nem** beleértve) minden k -adik eleméből képzett listát (tuple-t vagy sztringet) az `l[n:m:k]` kifejezéssel kaphatjuk meg.
- Negatív k érték esetén n -től a lista (tuple vagy sztring) eleje felé lépkedünk.
- Ha a kezdő és végpont nem a haladási iránynak megfelelően van megadva, üres listát (tuple-t vagy sztringet) kapunk.
- k (és a második kettőspont is) elhagyható, ilyenkor az alapértelmezett értéke 1.
- n elhagyható, ilyenkor az alapértelmezett értéke a haladási iránynak megfelelően olyan, hogy a listát (tuple-t vagy sztringet) a legelső (legutolsó) elemétől kapjuk meg.
- m elhagyható, ilyenkor az alapértelmezett értéke a haladási iránynak megfelelően olyan, hogy a listát (tuple-t vagy sztringet) a legutolsó (legelső) eleméig megkapjuk.

Szeletek (slice)

list, tuple és str típusok esetében

- Példák

```
>>> l = [n for n in range(-4,5)]
>>> l[::]
[-4, -3, -2, -1, 0, 1, 2, 3, 4]
>>> l[3]      # Ez index, nem szelet!
-1
>>> l[3:7]
[-1, 0, 1, 2]
>>> l[2:7]
[-2, -1, 0, 1, 2]
>>> l[2:7:2]
[-2, 0, 2]
```

```
>>> l[:-3:]
[-4, -3, -2, -1, 0, 1]
>>> l[-3:2:-2]
[2, 0]
>>> l[0:len(l):-1]
[]

>>> ('nulla', 'egy', 'kettő')[-2:]
('egy', 'kettő')
>>> 'Az_nem_menza'[::-1]
'aznem men zA'
```


A range (*intervallum*) adattípus

- Egy egész számokból álló számtani sorozatot reprezentál.
 - A range egy úgynevezett generátor típus, az elemei nem előre tárolódnak, hanem szükség esetén egyesével generálódnak.

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(-10))
[]
>>> list(range(-5, 6))
[-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]
>>> list(range(-5, 6, 4))
[-5, -1, 3]
>>> list(range(-5, 6, -4))
[]
>>> list(range(6, -5, -1))
[6, 5, 4, 3, 2, 1, 0, -1, -2, -3, -4]
```

A str, list és tuple adattípusok bejárása

Elemek szerint

- A ciklussal sorban megkapjuk a listaszerű objektum elemeit, de azt nem fogjuk tudni, hogy az adott elemnek mi volt az indexe (hacsak külön nem számoljuk).

```
>>> listlike = ["Egy", 2, (3, 4)]
>>> for e in listlike:
...     print(f"{listlike[e]!r}")
...
'Egy'
2
(3, 4)
```

A str, list és tuple adattípusok bejárása

Indexek szerint

- A ciklussal sorban megkapjuk a listaszerű objektum elemeinek az indexeit. Az index segítségével esetleg magához az elemhez is hozzáférhetünk.

```
>>> listlike = ["Egy", 2, (3, 4)]
>>> for i in range(len(listlike)):
...     f"A konténer {i}. eleme: {listlike[i]!r}"
...
A konténer 0. eleme: 'Egy'
A konténer 1. eleme: 2
A konténer 2. eleme: (3, 4)
```

A str, list és tuple adattípusok bejárása

Index-elem párok szerint

- A ciklussal sorban megkapjuk a listaszerű objektum indexeit és az adott indexeknek megfelelő elemeit is.

```
>>> listlike = ["Egy", 2, (3, 4)]
>>> for i, e in enumerate(listlike):
...     f"A konténer {i}. eleme: {e!r}"
...
A konténer 0. eleme: 'Egy'
A konténer 1. eleme: 2
A konténer 2. eleme: (3, 4)
```

Tartalomjegyzék

- 
- 1 Bemutakozás**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szkevenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő**
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Parancssori argumentumok kezelése

- A Python nyelvet támogató környezetekben lehetőség van arra, hogy a végrehajtás megkezdésekor a programnak parancssori argumentumokat adjunk át.
- A programunkban ezeket az argumentumokat egy listában, sztringekként kapjuk meg.
- A listát a `sys` modul `argv` változóján keresztül érjük el.
- A lista 0. indexű eleme (`sys.argv[0]`) a programunk neve.
- A program valódi argumentumai az 1. indexen kezdődnek.

```
1#!/usr/bin/env python3
2import sys
3print(sys.argv)
```

```
$ ./program.py alma körte meggy
['./program.py', 'alma', 'körte', 'meggy']
$ python3 args.py egy 2 3.0e0
['args.py', 'egy', '2', '3.0e0']
```

A basename program

A `.split()` segítségével

- Egy útvonal basename-je meghatározható, ha az útvonalat a könyvtárelválasztóknál daraboljuk, és a lista utolsó elemét vesszük.



Basename (a .split() használatával)

basename_split.py [1–21]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2025-09-16
6cc-by-nc-nd
7
8Adott egy (linuxos) útvonalleírás, nyerjük ki belőle
9a fájlnevét (az útvonal nélkül).
10"""
11from sys import argv
12
13def basename(path):
14    parts = path.split('/')
15    return parts[-1]
16
17def main():
18    print(basename(argv[1]))
19
20if __name__ == "__main__":
21    main()
```


Eltelt idő kiszámítása

Algoritmustervezés

- Kezeljük az időpontot egyetlen (összetett) értéként!
 - Az időpontot egy `tuple` típusú érték fogja reprezentálni.
 - A beolvasást és a konverziókat függvények segítségével oldjuk meg.



Eltelt idő [1/2]

eltelt_tuple.py [1-18]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Egy nap két időpontja között eltelt idő kiszámítása
9"""
10def input_ido(msg):
11    print(msg)
12    o = int(input("óra: "))
13    p = int(input("perc: "))
14    return (o, p)
15
16def ido_to_str(t):
17    (o, p) = t
18    return "%dóra %dperc" % (o, p)
```

Eltelt idő [2/2]

eltelt_tuple.py [20–39]

```
20 def ido_to_int(t):
21     (ora, perc) = t
22     return 60 * ora + perc
23
24 def int_to_ido(t):
25     return (t // 60, t % 60)
26
27 def eltelt_ido(t1, t2):
28     return int_to_ido(ido_to_int(t2) - ido_to_int(t1))
29
30 def main():
31     t1 = input_ido("Kérem az első időpontot!")
32     t2 = input_ido("Kérem a második időpontot!")
33
34     dt = eltelt_ido(t1, t2)
35
36     print("Az eltelt idő: %s." % (ido_to_str(dt)))
37
38 if __name__ == "__main__":
39     main()
```

Eltelt idő kiszámítása

Algoritmustervezés

- Vegyük az időpontokat a parancssorból!
 - Az időpontot egy `tuple` típusú érték fogja reprezentálni.
 - A konverziókat függvények segítségével oldjuk meg.



Eltelt idő [1/2]

eltelt_arg.py [1-18]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Egy nap két időpontja között eltelt idő kiszámítása
9"""
10from sys import argv
11
12def str_to_ido(t):
13    (o, p) = t.split(':')
14    return (int(o), int(p))
15
16def ido_to_str(t):
17    (o, p) = t
18    return "%dóra %dperc" % (o, p)
```

Eltelt idő [2/2]

eltelt_arg.py [20–37]

```
20 def ido_to_int(t):
21     (ora, perc) = t
22     return 60 * ora + perc
23
24 def int_to_ido(t):
25     return (t // 60, t % 60)
26
27 def eltelt_ido(t1, t2):
28     return int_to_ido(ido_to_int(t2) - ido_to_int(t1))
29
30 def main():
31     t1 = str_to_ido(argv[1])
32     t2 = str_to_ido(argv[2])
33     dt = eltelt_ido(t1, t2)
34     print("Az eltelt idő: %s." % (ido_to_str(dt)))
35
36 if __name__ == "__main__":
37     main()
```